

DEBRIEF

AI-Powered Meeting Notes

Master Architecture Rubric & SaaS Development Blueprint

Version v1.0 → SaaS v2.0	Status Active Development	Priority Builds Rooms · Cloud Sync · Offline · Frontend↔Backend Link	Stack React + TypeScript · Node.js · Express · Prisma · SQLite → PostgreSQL
--	---	--	---

How to Use This Document

This rubric is designed to be injected directly into another LLM as a system prompt or context block. It gives that model the complete picture of the Debrief codebase — every file, every current state, every exact change needed — so it can implement production-ready SaaS features without asking clarifying questions.

 **PROMPT TEMPLATE** Paste everything from Section 1 onward, then append: "Implement [FEATURE NAME] exactly as described in the rubric. Do not summarize or explain — write all changed files in full."

This document covers five areas:

- Section 1 — Complete codebase inventory (every file, what it does, what it currently uses)
- Section 2 — Your two priority features: Rooms and Cloud Sync, with exact implementation specs
- Section 3 — Database schema changes required
- Section 4 — Frontend↔Backend integration layer (API service, auth, offline support)
- Section 5 — SaaS roadmap with phased milestones

Section 1 — Complete Codebase Inventory

This is the exact state of all files as of the last build. Every LLM working on this project must treat this as the ground truth.

1A · Frontend (React + TypeScript + Vite + Tailwind v4)

Root: debrief-responsive/ | Dev server: http://localhost:5173 | Build: npm run build

Entry & Config

Component	Current State	Needed Change	Priority
src/main.tsx	App entry point	Wrap with new ApiProvider, SyncProvider	HIGH
src/types.ts	TypeScript interfaces for Note, AuthUser, Route	Add Room, Member, SyncStatus types	HIGH
src/index.css	Tailwind v4 @theme + base styles	No change needed	LOW
vite.config.ts	Vite bundler config	Add PWA plugin (vite-plugin-pwa)	HIGH
index.html	HTML shell	Add <link rel=manifest>	MED

Context (State Management)

Component	Current State	Needed Change	Priority
src/context/AuthContext.tsx	Local-only fake auth (any email/password works)	Replace with real JWT auth against backend /auth/login	CRITICAL

Component	Current State	Needed Change	Priority
src/context/NotesContext.tsx	localStorage-only CRUD, seeded with hardcoded data	Add API sync layer: optimistic local write → POST /notes → cloud confirm	CRITICAL
src/context/RouterContext.tsx	Custom useState-based router	Add /rooms/:id route	MED

Components

Component	Current State	Needed Change	Priority
src/components/Sidebar.tsx	Nav links: All/Action/Decision/Discussion/Problem + New Note + user avatar + logout	Add "Rooms" section below Views with room list	HIGH
src/components/TopBar.tsx	Breadcrumb + search input. Hamburger on mobile	Add sync status indicator (cloud icon: syncing/synced/offline)	MED
src/components/NoteCard.tsx	Expandable card: badge, title, content, share, delete, metadata chips	No change needed	LOW
src/components/NoteList.tsx	Responsive 1–2 col grid with AnimatePresence	No change needed	LOW
src/components/NoteInput.tsx	Modal sheet (slides up on mobile, scales on desktop). Calls categorize.ts	Wire onSave to POST /notes API instead of local state only	CRITICAL
src/components/GroupChat.tsx	Floating chat panel with hardcoded mock messages and fake bot reply	Replace with real WebSocket (socket.io) connected to /chat. Replace mock with real message history.	CRITICAL
src/components/SummaryPanel.tsx	Live counts sidebar (desktop only)	No change needed	LOW
src/components/BottomNav.tsx	Mobile bottom nav with FAB	Add Rooms tab	MED

Pages

Component	Current State	Needed Change	Priority
src/pages/LoginPage.tsx	Email + password form. Calls local AuthContext.login() which accepts anything	POST to /auth/login, store JWT in localStorage, set auth header for all future requests	CRITICAL

Services & Hooks

Component	Current State	Needed Change	Priority
src/services/categorize.ts	Keyword-based classifier (regex scoring). Returns { category, title, priority, assignee, dueDate }	Keep as-is for offline classification. Backend also classifies — use backend result when online.	LOW
src/hooks/useBreakpoint.ts	Returns "mobile" "tablet" "desktop" based on window.innerWidth	No change needed	LOW

Missing Frontend Files (Must Create)

Component	Current State	Needed Change	Priority
src/services/api.ts	Does not exist	Create: central Axios/fetch wrapper. Sets Authorization header from stored JWT automatically on every request.	CRITICAL
src/services/syncQueue.ts	Does not exist	Create: offline queue. If network down, store note in IndexedDB queue. Replay when back online.	HIGH
src/context/RoomContext.tsx	Does not exist	Create: room state — active room, member list, room notes fetched from GET /rooms/:id	HIGH
src/pages/RoomPage.tsx	Does not exist	Create: split view — left: room notes (read-only feed), right: live group chat. Joinable via share link.	HIGH
src/components/RoomCard.tsx	Does not exist	Create: room list item showing room name, member avatars, last activity	MED
src/hooks/useSocket.ts	Does not exist	Create: socket.io-client hook. Connects to ws://localhost:3001 with JWT auth. Handles room:join, note:new, chat:message events.	HIGH
src/hooks/useOfflineSync.ts	Does not exist	Create: reads IndexedDB queue on reconnect, replays pending writes to backend.	HIGH
public/manifest.json	Does not exist	Create: PWA manifest for installability and offline splash.	MED
public/sw.js	Does not exist	Auto-generated by vite-plugin-pwa. Caches notes for offline read.	MED

1B · Backend (Node.js + Express + Prisma + SQLite)

Root: debrief-backend/ | Server: http://localhost:3001 | Start: npm run dev

Entry & Config

Component	Current State	Needed Change	Priority
src/server.js	Express app: cors, json parser, request logger, route registration, 404+error handlers	Add socket.io upgrade alongside HTTP server. Add /rooms routes.	HIGH
prisma/schema.prisma	4 models: User, Meeting, Note, ChatMessage	Add: Room, RoomMember, RoomInvite. Migrate Note to add roomId (optional).	CRITICAL
prisma/seed.js	Creates demo user + meeting + 4 notes + 2 chat messages	Add demo room seeding	LOW
.env	DATABASE_URL, JWT_SECRET, PORT=3001, FRONTEND_URL	Add: SOCKET_CORS_ORIGIN	LOW

Middleware

Component	Current State	Needed Change	Priority
src/middleware/auth.js	Reads Authorization: Bearer <token>, verifies JWT, sets req.user	No change needed. Also needs socket middleware for socket.io auth.	MED

Services

Component	Current State	Needed Change	Priority
src/services/classifyNote.js	Keyword classifier — same logic as frontend categorize.ts. Returns type + matched keywords.	No change needed. Already modular.	LOW

Controllers

Component	Current State	Needed Change	Priority
src/controllers/authController.js	register() → bcrypt hash + JWT. login() → bcrypt compare + JWT. getMe() → return user.	No change needed	LOW
src/controllers/meetingsController.js	createMeeting, getMeetings, getMeetingById (with notes+chat), deleteMeeting	No change needed	LOW
src/controllers/notesController.js	createNote (auto-classifies), getNotes (?type= ?meetingId= filters), getNotesByMeeting (grouped), deleteNote, reclassifyNote	Add: broadcastNoteToRoom() call after createNote when note has roomId	HIGH
src/controllers/chatController.js	sendMessage (POST /chat), getMessages (GET /chat/:meetingId)	Replace with socket.io event — REST fallback only	HIGH

Routes

Component	Current State	Needed Change	Priority
src/routes/auth.js	POST /auth/register, POST /auth/login, GET /auth/me	No change needed	LOW
src/routes/meetings.js	POST/GET /meetings, GET/DELETE /meetings/:id	No change needed	LOW
src/routes/notes.js	POST/GET /notes, GET /notes/meeting/:id, DELETE, PATCH	No change needed	LOW
src/routes/chat.js	POST /chat, GET /chat/:meetingId	Keep as fallback. Primary = socket.io events	MED

Missing Backend Files (Must Create)

Component	Current State	Needed Change	Priority
src/routes/rooms.js	Does not exist	Create: POST /rooms, GET /rooms, GET /rooms/:id, POST /rooms/:id/join, POST /rooms/:id/invite, DELETE /rooms/:id	CRITICAL
src/controllers/roomsController.js	Does not exist	Create: createRoom, getRooms, getRoomById (notes+members+chat), joinRoom (via invite code), inviteToRoom, deleteRoom	CRITICAL
src/services/socketService.js	Does not exist	Create: socket.io server setup. Events: room:join, room:leave, note:new, chat:message, note:reclassify. Auth via JWT handshake.	CRITICAL
src/services/offlineSync.js	Does not exist	Create: endpoint POST /sync/push — accepts array of queued offline operations, replays them in order, returns results.	HIGH
src/routes/sync.js	Does not exist	Create: POST /sync/push route	HIGH

Section 2 — Priority Features (Implement These First)

Feature A — Team Rooms (Share & Connect)

A Room is a shared space for a team. It has its own note feed (visible to all members), its own live group chat, and a shareable invite link. Anyone with the link or code can join. Notes created inside a room are visible to all room members in real time.

SCOPE One room = one team context. Notes added to a room are synced to all active members via WebSocket. Room persists on the server. Members can view all notes including ones added while they were offline (fetched on join).

Backend — Rooms Controller (`src/controllers/roomsController.js`)

Implement these 6 functions:

- `createRoom(req, res)`
 - Input: `{ name, description? }` → creates Room, sets creator as first RoomMember with role = "admin"
 - Generates a random 8-char invite code (e.g. "XKCD-7291") stored on the Room record
 - Returns: `{ room: { id, name, inviteCode, memberCount: 1 } }`
- `getRooms(req, res)`
 - Returns all rooms the current user is a member of
 - Include: member count, last note createdAt, last chat message text
- `getRoomById(req, res)`
 - Returns the full room: `{ room, members: [{id, name, role}], notes: [...], recentMessages: [...last 50] }`
 - This is the initial payload when a user opens a room
- `joinRoom(req, res)`
 - Input: `{ inviteCode }` → find Room by inviteCode
 - If already a member: return 200 with existing membership
 - Otherwise: create RoomMember record, return `{ room, role: "member" }`
- `inviteToRoom(req, res)`
 - Input: `{ email }` → look up user, create RoomMember record
 - Only room admin can invite
- `deleteRoom(req, res)`
 - Only room admin can delete. Cascade deletes members.

Backend — Prisma Schema Additions (`prisma/schema.prisma`)

Add these three models after the existing ChatMessage model:

```
model Room {
```

```

id          Int          @id @default(autoincrement())
name        String
description String?
inviteCode  String       @unique // 8-char code like "XKCD-7291"
createdAt   DateTime     @default(now())
createdById Int
createdBy   User          @relation(fields: [createdById], references: [id])
members     RoomMember[]
notes       Note[]        // notes created inside this room
}

```

```

model RoomMember {
  id          Int          @id @default(autoincrement())
  role        String       @default("member") // "admin" | "member"
  joinedAt    DateTime     @default(now())
  userId      Int
  user        User          @relation(fields: [userId], references: [id])
  roomId      Int
  room        Room          @relation(fields: [roomId], references: [id], onDelete: Cascade)
  @@unique([userId, roomId]) // one record per user per room
}

```

Also add to existing Note model:

```

roomId  Int? // null = private note, set = room note
room    Room? @relation(fields: [roomId], references: [id])

```

Also add to existing User model:

```

rooms      RoomMember[]
createdRooms Room[]

```

Backend — Socket Service (src/services/socketService.js)

Create this file and call it from server.js. It handles real-time events:

- On connection: verify JWT from handshake.auth.token — disconnect if invalid
- Event room:join → socket joins the socket.io room (identified by "room-{roomId}")
- Event chat:send → save ChatMessage to DB, emit chat:message to everyone in that room
- Event note:new → (triggered by notesController after POST /notes) emit to room members
- Event room:leave → socket leaves the socket.io room
- On disconnect → emit presence:left to room members

Backend — Routes (src/routes/rooms.js)

Wire these routes (all require requireAuth middleware):

```

POST  /rooms      → createRoom
GET   /rooms      → getRooms
GET   /rooms/:id  → getRoomById
POST  /rooms/join  → joinRoom   (body: { inviteCode })
POST  /rooms/:id/invite → inviteToRoom
DELETE /rooms/:id  → deleteRoom

```


Frontend — Room Page (src/pages/RoomPage.tsx)

This is the main UI for a room. Layout:

- Left panel (60% width): NoteList filtered to roomId. Header shows room name + member avatars. Includes NoteInput that posts to this room.
- Right panel (40% width): Chat panel — identical to GroupChat.tsx but using real socket.io messages instead of mock data.
- Top: "Share Room" button that copies the invite link (format: `https://yourapp.com/join/{inviteCode}`)
- On mount: call GET `/rooms/:id` to load initial data, then connect socket and join room:`{id}`

Frontend — Context (src/context/RoomContext.tsx)

Manage room state with these values:

```
interface RoomContextValue {  
  rooms: Room[];           // all rooms the user belongs to  
  activeRoom: Room | null;  // currently open room  
  roomNotes: Note[];        // notes in activeRoom  
  roomMessages: ChatMsg[];  // chat in activeRoom  
  loadRoom: (id: number) => Promise<void>;  
  joinRoom: (inviteCode: string) => Promise<void>;  
  createRoom: (name: string) => Promise<Room>;  
  sendMessage: (text: string) => void; // via socket  
}
```

Frontend — useSocket Hook (src/hooks/useSocket.ts)

Wraps socket.io-client. Usage: `const socket = useSocket()`

- Connects to `process.env.VITE_API_URL` with `{ auth: { token: localStorage.getItem("token") } }`
- Reconnects automatically on disconnect
- Exposes: `socket` (the instance), `connected` (boolean), `emit()`, `on()`
- Cleans up listeners on component unmount

Feature B — Cloud Sync + Offline Support

Notes are created locally first (optimistic UI — instant feedback). They are then synced to the backend. If the user is offline, writes go into an IndexedDB queue. When the user comes back online, the queue replays automatically. Notes are also cached for offline reading.

USER EXPERIENCE GOAL The user should never feel the network. Create a note → it appears instantly. Go offline → app still works. Come back online → everything syncs silently. A small status icon in the TopBar shows sync state.

API Service (src/services/api.ts) — CREATE THIS FILE

Central HTTP client. Every API call goes through this. Do not use raw fetch() anywhere else.

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? "http://localhost:3001",
  timeout: 10000,
});

// Auto-attach JWT to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem("debrief_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auto-logout on 401
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem("debrief_token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);

export default api;
```

Auth Context Rewrite (src/context/AuthContext.tsx)

Replace the fake login with real backend calls:

- login(email, password): POST /auth/login → get { user, token } → save token to localStorage("debrief_token") → save user to state
- register(name, email, password): POST /auth/register → same flow
- logout(): clear localStorage("debrief_token") and user state
- On app load (useEffect): if token exists, call GET /auth/me to validate it. If 401, clear and redirect to login.
- AuthUser type must include id: number — needed for socket auth and ownership checks

Notes Context Rewrite (src/context/NotesContext.tsx)

Replace localStorage-only CRUD with a sync layer:

- On mount: call GET /notes to load all notes from server. Store in state AND write to localStorage as cache.
- addNote(content, meetingId?, roomId?):

- 1. Immediately add to local state with status: "pending" and a temp UUID
- 2. Call POST /notes { content, meetingId, roomId }
- 3. On success: replace the temp note with the real one from server (real ID, real classification)
- 4. If offline: add to syncQueue instead — local state still updates immediately
- deleteNote(id): call DELETE /notes/:id, remove from local state on success
- syncStatus: "synced" | "syncing" | "offline" — exposed to components for the TopBar indicator

Offline Queue (src/services/syncQueue.ts) — CREATE THIS FILE

Uses IndexedDB (via the idb library) to queue writes when offline:

```
interface QueuedOperation {
  id: string;          // temp UUID
  type: "CREATE_NOTE" | "DELETE_NOTE" | "SEND_MESSAGE";
  payload: unknown;    // the request body
  timestamp: number;
}
```

- enqueue(op): saves to IndexedDB store "sync_queue"
- getAll(): returns all queued ops, sorted by timestamp
- remove(id): deletes one op after successful replay
- clear(): wipes all queued ops (used on full sync)

Offline Sync Hook (src/hooks/useOfflineSync.ts) — CREATE THIS FILE

Runs in the background. Watches for network reconnection and replays the queue:

```
window.addEventListener("online", async () => {
  const ops = await syncQueue.getAll();
  for (const op of ops) {
    try {
      await api.post("/sync/push", op);
      await syncQueue.remove(op.id);
    } catch { break; } // stop on first failure
  }
});
```

Backend Sync Endpoint (POST /sync/push)

Accepts a single queued operation from the frontend and replays it:

```
POST /sync/push
Body: { type: "CREATE_NOTE", payload: { content, meetingId, roomId } }
→ runs the same logic as POST /notes
→ returns { success: true, result: <created note> }
```

PWA Setup (vite.config.ts + public/manifest.json)

Makes the app installable and cacheable for offline use:

- Install vite-plugin-pwa: `npm install -D vite-plugin-pwa`
- In vite.config.ts: add `VitePWA({ registerType: "autoUpdate", strategies: "generateSW" })`
- Workbox will auto-cache: all built JS/CSS/HTML, all API GET responses for /notes and /rooms
- manifest.json: `{ name: "Debrief", short_name: "Debrief", start_url: "/", display: "standalone", background_color: "#F7F6F3" }`

TopBar Sync Indicator (src/components/TopBar.tsx)

Small icon next to the search bar showing current sync state:

- "synced" →  cloud icon in gray (all notes uploaded)
- "syncing" →  spinning icon in blue (upload in progress)
- "offline" →  icon in amber (no connection, using cache)
- Reads syncStatus from NotesContext

Section 3 — Database Schema (Full Updated Schema)

Run these commands after updating schema.prisma:

```
npx prisma migrate dev --name add_rooms_and_sync
npx prisma generate
```

IMPORTANT Do NOT delete and recreate the database. Use migrations so existing notes and users are preserved. The only new column on Note is roomId (optional/nullable) — it does not break existing rows.

Changes Summary

Component	Current State	Needed Change	Priority
User model	Existing: id, name, email, password, createdAt	Add: rooms RoomMember[], createdRooms Room[]	CRITICAL
Meeting model	No change needed	—	LOW
Note model	Existing: id, content, type, createdAt, meetingId, authorId	Add: roomId Int? + room Room? relation	HIGH
ChatMessage model	Existing: id, text, createdAt, meetingId, senderId	Add: roomId Int? for room-level chat	HIGH
Room model	Does not exist	Create: id, name, description, inviteCode, createdAt, createdById	CRITICAL
RoomMember model	Does not exist	Create: id, role, joinedAt, userId, roomId + @@unique([userId,roomId])	CRITICAL

Section 4 — Frontend ↔ Backend Integration Map

Every place in the frontend that currently uses localStorage or mock data, and what it should call instead:

Authentication Flow

Component	Current State	Needed Change	Priority
LoginPage.tsx → AuthContext.login()	Calls fake login that accepts anything	POST /auth/login → { token, user } → localStorage.setItem("debrief_token", token)	CRITICAL
App load (AuthContext useEffect)	No token validation on startup	GET /auth/me with stored token. If 401 → logout.	CRITICAL
All API requests (api.ts interceptor)	No auth header added	Auto-add: Authorization: Bearer <token> to every request	CRITICAL

Notes Flow

Component	Current State	Needed Change	Priority
NotesContext — initial load	INITIAL_NOTES hardcoded array	GET /notes on mount. Fallback to localStorage cache if offline.	CRITICAL
NoteInput.tsx → onSave	Calls addNote() → localStorage only	addNote() does: (1) optimistic local update (2) POST /notes (3) replace with server response	CRITICAL
NoteCard.tsx → handleDelete	Calls deleteNote() → localStorage only	deleteNote() → DELETE /notes/:id then remove from state	HIGH
NoteCard.tsx → handleShare	Copies raw text to clipboard	No change needed — this is fine	LOW

Chat Flow

Component	Current State	Needed Change	Priority
GroupChat.tsx — messages	Hardcoded MOCK_MESSAGES array	On open: GET /chat/:meetingId or GET /rooms/:roomId for initial messages	CRITICAL
GroupChat.tsx — send	Pushes to local state + fake bot reply after 1.8s	socket.emit("chat:send", { text, roomId }) — socket broadcasts to all members	CRITICAL
GroupChat.tsx — receive	No real-time receive	socket.on("chat:message", msg => setMessages(prev => [...prev, msg]))	CRITICAL

Rooms Flow

Component	Current State	Needed Change	Priority
No rooms feature exists yet	—	Add "Rooms" to Sidebar. Clicking opens RoomPage. Create/join flow via invite code.	HIGH
Share invite link	—	Room invite link format: {VITE_APP_URL}/join/{invite Code}. Copy to clipboard.	HIGH
Deep-link join	—	If URL is /join/:code → call POST /rooms/join { inviteCode } → redirect to /rooms/:id	MED

Environment Variables

Add a .env.local file in the frontend root (never commit this):

```
VITE_API_URL=http://localhost:3001
VITE_SOCKET_URL=http://localhost:3001
VITE_APP_URL=http://localhost:5173
```

New npm Packages Required

Frontend — run: npm install

- axios — HTTP client (replaces raw fetch everywhere)
- socket.io-client — real-time socket connection
- idb — IndexedDB wrapper for offline queue
- vite-plugin-pwa — PWA + service worker generation (devDependency)

Backend — run: npm install

- socket.io — WebSocket server (integrates with existing Express server)

Section 5 — SaaS Roadmap (Phased Milestones)

Phase 1 — Working Product (Current Sprint)

Everything needed for a real, usable MVP. Do these in order.

- [] Create src/services/api.ts — central HTTP client with JWT auto-attach
- [] Rewrite AuthContext — real login/register via backend
- [] Rewrite NotesContext — load from GET /notes, save to POST /notes
- [] Wire NoteInput to POST /notes API
- [] Add Room + RoomMember to prisma/schema.prisma + migrate
- [] Create roomsController.js + rooms.js routes
- [] Create src/services/socketService.js in backend
- [] Create src/hooks/useSocket.ts in frontend
- [] Create RoomPage.tsx (notes feed + chat panel)
- [] Replace GroupChat mock with real socket.io
- [] Add Rooms to Sidebar navigation
- [] Create syncQueue.ts + useOfflineSync.ts + POST /sync/push
- [] Add vite-plugin-pwa + manifest.json

Phase 2 — SaaS Foundations

- [] Upgrade SQLite → PostgreSQL (change DATABASE_URL + migrate)
- [] Deploy backend to Railway / Render (free tier is fine)
- [] Deploy frontend to Vercel (set VITE_API_URL to deployed backend URL)
- [] Add rate limiting (express-rate-limit) — 100 req/15min per IP
- [] Add email verification on register (nodemailer + token)
- [] Add password reset flow (email link with short-lived token)
- [] Role-based permissions (admin vs member in rooms)

Phase 3 — Growth Features

- [] Export meeting as PDF (jspdf + html2canvas in frontend)
- [] Search across all notes (GET /notes?q=query)
- [] Note reactions / upvotes
- [] Multiple workspaces per user (Workspace model — org-level container)
- [] Mention @username in chat → notification
- [] AI classification upgrade (swap classifyNote.js for OpenAI API call)

Phase 4 — Monetization

- [] Stripe integration — subscription plans (free: 3 rooms, pro: unlimited)
- [] Usage limits enforced in backend (check plan before createRoom)
- [] Billing portal page (Stripe Customer Portal)
- [] Team management (invite by email, manage members, transfer admin)
- [] Audit log (who created/deleted what, when)
- [] Analytics dashboard (notes per meeting, most active rooms, team activity)

LLM Prompt Templates

Copy-paste these when working with another LLM. Paste this entire document first, then use one of these prompts.

Prompt A — Implement Rooms Feature

You are a senior full-stack engineer. I have given you the complete rubric for the Debrief app above. Implement Feature A (Team Rooms) exactly as described in Section 2A. Write the complete file contents for: `prisma/schema.prisma` (updated), `src/controllers/roomsController.js`, `src/routes/rooms.js`, `src/services/socketService.js`, `src/context/RoomContext.tsx`, `src/pages/RoomPage.tsx`, `src/hooks/useSocket.ts`. Do not summarize. Write every file in full.

Prompt B — Implement Cloud Sync + Offline

You are a senior full-stack engineer. I have given you the complete rubric for the Debrief app above. Implement Feature B (Cloud Sync + Offline) exactly as described in Section 2B. Write the complete file contents for: `src/services/api.ts`, `src/context/AuthContext.tsx` (rewritten), `src/context/NotesContext.tsx` (rewritten), `src/services/syncQueue.ts`, `src/hooks/useOfflineSync.ts`, `src/components/TopBar.tsx` (updated with sync indicator), `vite.config.ts` (with PWA plugin), `public/manifest.json`, backend: `src/routes/sync.js` + `src/services/offlineSync.js`. Do not summarize. Write every file in full.

Prompt C — Link Frontend to Backend (Auth Only)

You are a senior full-stack engineer. I have given you the complete rubric for the Debrief app above. Wire the frontend authentication to the real backend as described in Section 4 (Authentication Flow). Write the complete file contents for: `src/services/api.ts`, `src/context/AuthContext.tsx`, `src/pages/LoginPage.tsx`. The backend is already built and running at `http://localhost:3001`. Do not summarize. Write every file in full.